AI COURSE – II AIML/REC

CATEGORY 1 – PYTHON PROGRAMS ONLY

1. Simple Reflex Agent

```
def simple_reflex_agent(location, status):
    if status == "Dirty":
        return "Suck"
    elif location == "A":
        return "Move Right"
    else:
        return "Move Left"


location = input("Enter location (A/B): ")
status = input("Enter room status (Clean/Dirty): ")

action = simple_reflex_agent(location, status)

print("Current Location:", location)
print("Room Status:", status)
print("Agent Action:", action)
```

2. Model Based Agent

```
class ModelBasedAgent:
    def __init__(self):
        self.model = {
            "A": "Unknown",
            "B": "Unknown"
        }

    def act(self, location, status):
        self.model[location] = status

        if status == "Dirty":
            return "Suck"
        elif self.model["A"] == "Unknown":
            return "Move to A"
        elif self.model["B"] == "Unknown":
            return "Move to B"
        else:
            return "All rooms are clean"


agent = ModelBasedAgent()

while True:
    location = input("Enter location (A/B): ")
    status = input("Enter status (Clean/Dirty): ")

    action = agent.act(location, status)

    print("Agent Action:", action)
    print("Internal Model:", agent.model)

    if action == "All rooms are clean":
        break
```

3. Goal Based Agent

```
class GoalBasedAgent:
    def __init__(self, goal):
        self.goal = goal

    def act(self, current_location):
        if current_location == self.goal:
            return "Goal Reached"
        return "Move Towards Goal"


goal = input("Enter destination (A/B): ")
current = input("Enter current location (A/B): ")

agent = GoalBasedAgent(goal)

while True:
    action = agent.act(current)
    print("Agent Action:", action)

    if action == "Goal Reached":
        break

    current = goal
    print("Current Location:", current)
```

4. Utility Based Agent

```
def utility_based_agent():
    utilities = {
        "Study": 80,
        "Play": 60,
        "Sleep": 90
    }

    print("Available Actions and Utilities:")

    for action, utility in utilities.items():
        print(action, ":", utility)

    best_action = max(utilities, key=utilities.get)

    print("Best Action:", best_action)
    print("Maximum Utility:", utilities[best_action])


utility_based_agent()
```